NATIONAL AUTONOMOUS UNIVERSITY OF MEXICO

FACULTY OF ENGINEERING

COMPUTER GRAPHICS AND HUMAN-COMPUTER INTERACTION

SEMESTER 2027 - 1

FINAL PROJECT

CEMPASÚCHIL

Ofrenda, memoria y tradición

TECHNICAL AND USER DOCUMENTATION

STUDENTS	*Basilio Illescas Andrés*	*Product Owner*
	Trejo Olvera Emmanuel	*Scrum Master*
	Pérez Olvera Alexis Abraham	*Developer*
	Cruz Macedo Samuel Santiago	*Developer*
PROFESSOR	*Eng. José Ramón Pérez Athié*	
GROUP	*04*	

Ciudad Universitaria, Mexico City

November 11, 2026

EVIDENCE SHEET

CEMPASÚCHIL: Offering, Memory, and Tradition

Final project, Computer Graphics and Human-Computer Interaction

Team 8 — Group 04

Student	Student ID (número de cuenta)
Basilio Illescas Andrés	321137135
Cruz Macedo Samuel Santiago	424065465
Pérez Olvera Alexis Abraham	321150503
Trejo Olvera Emmanuel	424138220

Professor: Eng. José Ramón Pérez Athié

Semester: 2027-1

Contents

List of Figures 4

List of Tables 5

Abstract 6

1 Introduction 6

2 Concept, Memory, and Meaning of the Ofrenda 7

2.1 Choosing Día de Muertos . . . 7

2.2 The Seven Portraits: Music, Fragility, and Permanence . . . 7

2.3 Translating the Concept into the Scene . . . 8

3 Objectives 8

3.1 General Objective . . . 8

3.2 Specific Objectives . . . 8

4 Practical Methodology 9

4.1 Prototype-Based Methodology . . . 9

4.2 Project Evolution across Deliveries . . . 9

4.2.1 Planning and a Prototype Made from Primitives . . . 9

4.2.2 From an Isolated Altar to a Complete Scene . . . 10

4.2.3 Reorganization and Integration . . . 11

4.2.4 Texturing, Color, and Lighting . . . 11

4.2.5 Animation and Final Integration . . . 13

4.3 Team Organization . . . 13

4.4 Tools and Technologies . . . 14

4.5 Modeling and Scene Composition . . . 14

4.6 Texturing, Color, and Lighting . . . 15

4.7 Animation 16
4.7.1 Code Animations 17
4.7.2 Animations Created in Blender 17
4.7.3 Skeletal Animations 18
4.8 Navigation and Interaction 19
4.9 Integration of the Photographs 20

5 Experiments **20**
5.1 Building on Different Platforms 20
5.2 Automation and Distribution of the Windows Executable 21
5.3 Version Control and Work Integration 21
5.4 Performance Problems and Optimization Decisions 22
5.5 Animation Validation 22
5.6 Performance 23

6 Results **23**

7 Visual Evidence of the Final Scene **26**

8 Demonstration Video **29**

9 General Conclusions **29**

10 Individual Conclusions **30**

11 References **33**

List of Figures

1 Initial prototype built from primitives, flat colors, and a basic three-tier composition. 10

2 Modeling stage: composition without materials (left) and review of polygonal construction in wireframe mode (right). 11

3 Texturing and lighting stage: view of the *ofrenda* and detail of materials using baked maps. 12

4 Test of translucent materials using a Fresnel approximation for glass and water. 12

5 General view: cemetery, petal path, and the house with the offering. 15

6 General views and model details. The route connects the entrance, path, and ofrenda. Close views show baked textures, normal maps, transparency, and the photographs on the altar. 27

7 Interactive lighting and animations programmed in OpenGL. Key L changes lanterns and votive candles. Butterflies combine Catmull–Rom with a body–wing hierarchy, while the automatic camera travels to the altar. 27

8 Animations created in Blender. Exported caches and tracks preserve the papel picado deformation, the petal whirlwind, and the pitcher motion with the changing water level. 28

9 Skeletal animations. The clips show the visitor placing a pan de muerto, the character walking through the cemetery, and the skeleton dancing beside the gateway. 28

10 CEMPASÚCHIL demonstration video. Click the image to open the video on YouTube. 29

List of Tables

1 Phases of the prototype-based methodology. 9
2 Main roles and responsibilities of the team. 13
3 Main iterations and the result of each increment. 14
4 Tools used. 14
5 Available animations and interactions. 16
6 Application controls. 19

Abstract

Cempasúchil is an interactive graphics application written in C++17 with OpenGL 3.3 and GLSL that recreates a *Día de Muertos* night. The visitor walks through a cemetery, follows a petal path lined with lanterns, votive candles, and *papel picado*, and reaches a three-tier *ofrenda*, a ritual altar offering, on the porch of an adobe house. The system covers the full graphics pipeline seen in the course: hierarchical modeling in Blender, glTF 2.0 interchange, texturing with color, roughness, and normal maps, Phong lighting with point lights, a Fresnel approximation for glass and water, a skybox, and nine keyboard-triggered animations. Three animations are programmed directly in OpenGL, three were created in Blender and are played back from exported tracks, and three use rigged characters whose clips are evaluated with linear blend skinning. The result meets the minimum requirements of the specification and builds on macOS, Windows, and Linux.

1 Introduction

The Day of the Dead is one of the most representative celebrations in Mexico. In 2008 UNESCO inscribed the indigenous festivity dedicated to the dead on the Representative List of the Intangible Cultural Heritage of Humanity (UNESCO, 2008). Its visual elements (the cempasúchil flower, papel picado, votive candles, the tiered offering, and the petal path that guides the souls) suit a graphics project well because they combine repeated geometry, translucent materials, small light sources, and motion.

The course specification asks for an interactive system that goes through every stage of the graphics pipeline: conceptualization, geometric modeling, color, texturing, lighting, shading, animation, interaction, and rendering with OpenGL 3 or later (Segal and Akeley, 2010). Shading uses the reflection model of Phong (1975), which separates ambient, diffuse, and specular terms, and the approximation by Schlick (1994) to the Fresnel term, so that glass and liquids reflect more at grazing angles. Models are exchanged in glTF 2.0 (Khronos Group, 2021), a format that keeps node hierarchies, PBR materials, skeletons, and animation clips.

For animation the project uses three families of techniques described in real-time rendering literature (Akenine-Möller et al., 2018; Gregory, 2018): basic transformations applied to a hierarchy, keyframe interpolation, and procedural animation. Trajectories are interpolated with Catmull-Rom splines (Catmull and Rom, 1974) in their centripetal variant, which avoids loops (Yuksel et al., 2011), and rotations with spherical quaternion interpolation (Shoemake, 1985). Rigged characters are deformed with per-vertex linear blend skinning, the technique introduced by Magnenat-Thalmann et al. (1988) and now the standard in glTF.

This report describes the planning, visual evolution, design decisions, technical problems, validation experiments, and results of the final version. The project did not begin as a finished scene: it started with an altar made from cubes, cylinders, spheres, and flat colors; grew into a complete environment; received textures, lighting, and materials; and finally incorporated navigation and nine interactive animations. Documenting this process explains not only what was delivered, but also why each decision was made.

2 Concept, Memory, and Meaning of the Ofrenda

2.1 Choosing Día de Muertos

The team chose *Día de Muertos* because it naturally brings together memory, journey, and visual representation. Rather than building an isolated altar viewed from a single position, the goal was to create an experience: entering through a cemetery, walking among graves, following light and petals, and gradually discovering the *ofrenda*. Navigation therefore became part of the narrative. The cempasúchil path guides the visitor; votive candles and lanterns make the route visible; *papel picado* introduces color and movement; monarch butterflies evoke return; and the altar gathers food, vessels, flowers, and portraits.

The scene avoids representing death only through darkness. The night, cemetery, and graves establish the setting, while orange flowers, warm light, moving petals, and offerings turn the space into a place of encounter. This duality guided the palette: cool blues and shadows along the path contrast with oranges, magentas, yellows, and warm illumination near the altar.

2.2 The Seven Portraits: Music, Fragility, and Permanence

The final version dedicates its seven picture frames to Janis Joplin, Kurt Cobain, Valentín Elizalde, Amy Winehouse, Jim Morrison, Jimi Hendrix, and Kristen Pfaff. All seven died at the age of 27. Their careers belong to different genres, countries, and generations, yet they share a creative intensity that survived lives cut short.

The intention is not to romanticize excess or portray suffering as a requirement for talent. Instead, the *ofrenda* recalls that behind these public figures were people facing pressure, isolation, addiction, violence, exhaustion, and expectations that often exceeded the support available to them. Valentín Elizalde also represents a Mexican musical tradition and a death marked by violence; Kristen Pfaff recognizes those who did not always occupy center stage but still left a decisive mark on their generation.

The altar makes a gesture of remembrance: it briefly separates these people from the myth of the "27 Club" and returns an individual place to each of them. Every photograph is different and occupies its own position. Water, *pan de muerto*, flowers, candles, food, and everyday objects surround them. Together, the scene suggests that a body of work may endure even when a life was short and difficult, and that remembrance also requires empathy for the human cost behind that work.

2.3 Translating the Concept into the Scene

The spatial arrangement reinforces the meaning. Portraits are distributed across all three tiers so that no hierarchy is imposed on the people being honored. The cempasúchil arch frames the composition; candles form lines and groups that direct the eye; water and containers make use of translucent materials; bread and food add volume and variety; and *papel picado* breaks the rigidity of the steps. One butterfly rests before the *ofrenda* and can take flight, while others appear along the path. The complete scene therefore becomes a transition from the silence of the cemetery to a memory illuminated and set in motion.

3 Objectives

3.1 General Objective

Develop an interactive OpenGL 3.3 graphics system that represents a Day of the Dead offering and applies the full graphics pipeline, from modeling to interactive animation.

3.2 Specific Objectives

- Integrate at least eight team-made models and eight library models, textured and organized in a hierarchical scene graph.
- Light the scene with at least two techniques (Phong for opaque materials and Fresnel for translucent ones) and an environment map.
- Provide at least two real-time light interactions.
- Integrate three code animations, three animations created in external software, and three skeletal animations, all triggered from the keyboard.
- Keep the CMake build working on macOS, Windows, and Linux.

4 Practical Methodology

4.1 Prototype-Based Methodology

The work followed the phases of the prototype-based methodology in the specification. Requirements gathering chose the theme, defined the interactions, and set the scope of each delivery. Conceptual design produced the scene layout (cemetery to the north, path in the middle, house and offering to the south) and the model catalog. From there a first prototype with gray geometry was built. At each delivery it was tested, corrected, and extended: first with complete modeling, then with textures and lighting, and finally with animation. Table 1 summarizes each phase.

Phase	Activity in the project
Requirements and planning	Theme “Mexico and its culture” with the Day of the Dead offering; list of models, lights, and minimum interactions.
Conceptual design	Scene plan, path axis as a spline, and first-person navigation flow.
Initial prototype	GLFW window, glTF loading, camera, and scene graph with untextured geometry.
Validation and feedback	Team review at each partial delivery; fixes to scale, intersections, and performance.
Iteration and improvement	PBR textures, point lights, light filtering, skybox, and animations.
Full implementation	Integration of the nine keyboard animations and documentation.

Table 1: Phases of the prototype-based methodology.

4.2 Project Evolution across Deliveries

4.2.1 Planning and a Prototype Made from Primitives

The first version deliberately had a small scope. A three-tier altar was built from cubes, while offerings were represented with primitives: cylinders for candles, spheres for fruit, a torus and cone for a simplified flower, and flat planes anticipating the portraits. Materials used flat colors and a warm light made the volumes readable. The prototype did not aim for realism; it answered composition questions such as the number and proportions of the tiers, the focal point, and the space future offerings would require.

Its main lesson was that a simple representation can reveal problems before time is invested in detail. Primitives made it possible to move complete groups, test framing, and confirm that the stepped altar provided enough surfaces to distribute objects without losing legibility.

Figure 1: Initial prototype built from primitives, flat colors, and a basic three-tier composition.

4.2.2 From an Isolated Altar to a Complete Scene

During the modeling delivery, the team concluded that the altar, although recognizable, was not enough to communicate an experience. The scope expanded into a walkable environment: an adobe house with a porch, a central path, and a cemetery at the opposite end. The route provided a natural place for lanterns, suspended *papel picado*, graves, crypts, walls, fences, vegetation, and butterflies.

The earliest team-made models were reviewed and remodeled where necessary. Scale, origins, normals, mesh density, and proportions were corrected. Some objects that worked in close-up became too expensive when repeated; others were too simple when placed in the foreground. The team adopted a definition of done for every model: applied transforms, an origin near the base, an identifiable name, correct normals, reasonable complexity, valid glTF export, and verification in both solid and wireframe views.

The minimum goal of eight original models was exceeded by dividing work into families. The team modeled the altar, flowers, petals, candles, *pan de muerto*, *calaveritas de azúcar*, *papel picado*, censer, picture frames, vases, vessels, butterflies, lanterns, architecture, food, sweets, and bottles. Environmental pieces from the *Kenney Graveyard Kit* complemented the cemetery without replacing authorship of the principal objects.

Figure 2: Modeling stage: composition without materials (left) and review of polygonal construction in wireframe mode (right).

4.2.3 Reorganization and Integration

As the catalog grew, organization became as difficult as modeling. Objects had to share a coherent scale, rest correctly on surfaces, and avoid occupying one another's space. The *ofrenda* was reorganized by zones and tiers; cells were reserved for groups of candles, food, portraits, and containers, and geometric checks prevented intersections. A Catmull–Rom curve became the common axis used to distribute petals and orient objects along the path.

This stage motivated the scene graph and instanced batches. A transform applied to a parent node moves all of its components, while thousands of repeated objects share a single geometry. This architecture allowed the scene to keep growing without turning every copy into an independent model.

4.2.4 Texturing, Color, and Lighting

The next delivery transformed the way the scene was read. Gray tones were replaced with materials distinguishing adobe, wax, paper, metal, ceramic, glass, water, flowers, and food. To avoid relying entirely on external photographs, many textures were designed procedurally in Blender. Noise networks, color blends, and relief were baked into conventional color, roughness, and normal maps that glTF and OpenGL could load efficiently.

Baking solved an important difference between Blender and the custom engine: a complex node network is not reproduced automatically in GLSL. Converting it into images preserved its overall appearance while controlling real-time cost. UVs, margins, color spaces, and normal-map orientation had to be corrected, and meshes whose density no longer provided visible detail were reduced.

Figure 3: Texturing and lighting stage: view of the *ofrenda* and detail of materials using baked maps.

Lighting was also redesigned. A cool directional light establishes the night, while votive candles and lanterns provide warm point lights. Grouped fill lights preserve the presence of the altar without evaluating every source against every object. Glass and water use Fresnel to strengthen their edges and transmit the background. The final decision was to keep all lighting in OpenGL: no static illumination was baked into textures, so switching, fading, and flickering remain interactive.

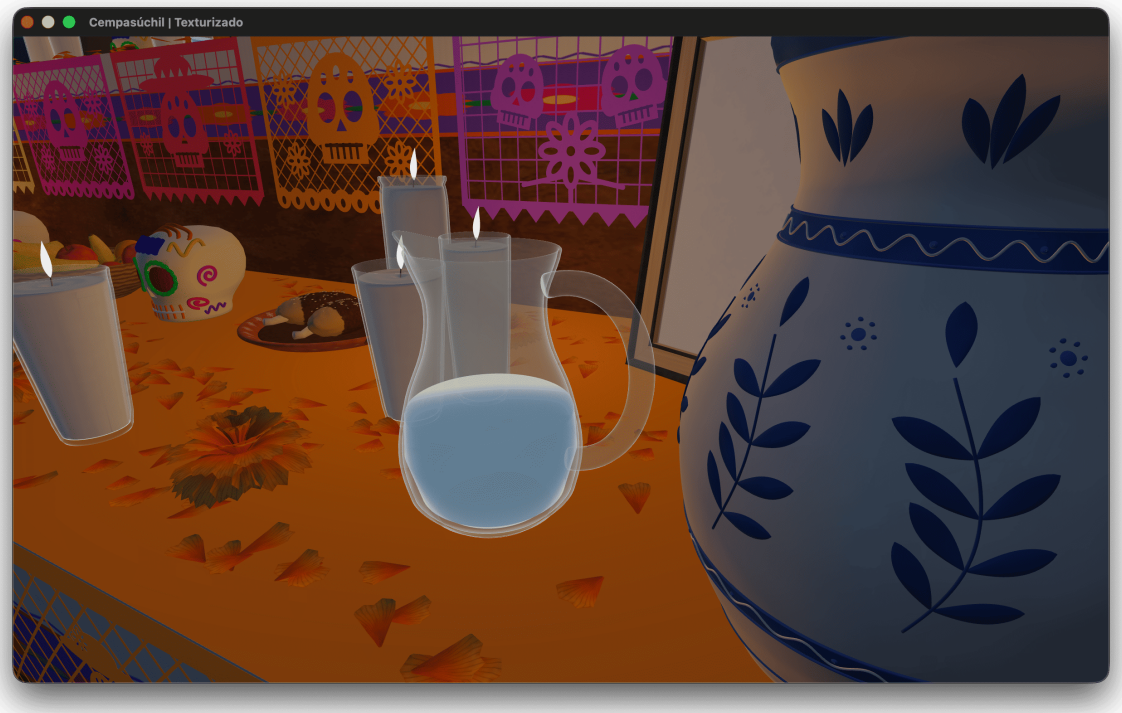


Figure 4: Test of translucent materials using a Fresnel approximation for glass and water.

4.2.5 Animation and Final Integration

With the scene modeled, textured, and lit, the final stage added movement without replacing the existing engine. Three code animations, three Blender animations, and three skeletal animations were produced. They were first developed as isolated clips or tests, then integrated into the scene, assigned to keys, adapted to existing hierarchies, and optimized so that light filtering and camera culling remained valid.

The most delicate task was reproducing external animations faithfully. Conventional export does not always preserve modifiers, simulations, or shape keys exactly as they appear in Blender. For that reason, *papel picado* was converted to a cache of per-vertex positions; the whirlwind and pitcher were exported as binary tracks; and characters were exported in glTF with an armature, skin, and clips. The final version plays these data inside OpenGL and combines them with the engine's interaction and lighting.

4.3 Team Organization

The team adapted Scrum to the four course deliveries. Each delivery worked as a sprint with an executable increment: prototype, modeling, texturing and lighting, and animation with product completion. The Product Backlog grouped infrastructure, modeling, composition, materials, animation, optimization, and documentation tasks. At the beginning of each iteration, tasks were prioritized by dependency and value to the scene. During development, the team communicated progress and blockers. Each sprint ended with a full review in OpenGL and a short retrospective.

Member	Role	Main responsibility
Basilio Illescas Andrés	Product Owner	Scope priorities, cultural validation, and increment reviews.
Trejo Olvera Emmanuel	Scrum Master	Coordination, Blender–OpenGL integration, performance, and compatibility.
Pérez Olvera Alexis Abraham	Developer	Ofrenda models, materials, textures, and glTF exports.
Cruz Macedo Samuel Santiago	Developer	Architecture, cemetery, route, and integration testing.

Table 2: Main roles and responsibilities of the team.

Work tracking used the states *pending*, *in development*, *in review*, *integrated*, and *done*. A task closed only after integration into the main scene and verification against its acceptance criteria. Model reviews covered scale, origin, normals, UVs, complexity, and export. Code reviews

required a clean build, portable paths, and no regressions. Animation reviews checked input activation, stable playback, and visual integration. Branches and focused commits separated changes, while `.gitignore` excluded build output and temporary files.

Sprint	Objective	Verifiable increment
1	Prototype	Concept, primitive altar, camera, and initial layout.
2	Modeling	Walkable environment, original models, hierarchies, and instances.
3	Appearance	Textures, skybox, Phong, Fresnel, lights, and initial optimization.
4	Animation and completion	Nine animations, navigation, testing, documentation, and packaging.

Table 3: Main iterations and the result of each increment.

The main risks involved platform differences, overly complex models, excessive light counts, external animations that were difficult to reproduce, and Git conflicts on binary files. Partial integrations, testing on macOS, Windows, and Linux, baked animation data, instancing, light filtering, and stable versions reduced these risks. This organization kept Blender, OpenGL, and documentation work connected instead of turning partial deliveries into separate projects.

4.4 Tools and Technologies

Tool	Use
C++17	Application, scene graph, interaction, and animation.
OpenGL 3.3 Core and GLSL 330	Rendering, materials, lighting, and plane clipping.
Blender 5.2 (Blender Foundation, 2026)	Modeling, texture baking, animation of the papel picado, petals, and pitcher, and character rigs.
glTF 2.0 and cgltf	Exchange of models, materials, skeletons, and clips.
GLFW, GLAD, and GLM	Window and input, OpenGL function loading, and math.
stb_image	PNG and JPEG decoding on Windows and Linux.
CMake	Cross-platform build configuration.

Table 4: Tools used.

4.5 Modeling and Scene Composition

The team-made models were built in Blender from primitives using extrusion, surfaces of revolution, and modifiers, and exported to glTF with their hierarchy intact. The altar, for instance, splits into its three tiers, and the lantern into post, arm, and lamp. The application replicates

that hierarchy as editable nodes: each node stores local translation, rotation, and scale, and its world matrix is $M_{\text{world}} = M_{\text{parent}}\, T\, R\, S$.

The final scene has 2,134 nodes, 81 loaded models, and 10,549 instances. Elements repeated thousands of times (petals, candles, flowers, and walls) are grouped in batches and drawn with `glDrawElementsInstanced`, with one matrix and one tint per instance. The path is a centripetal Catmull-Rom spline, evaluated with the algorithm of Barry and Goldman (1988) and reparameterized by arc length; it places petals, candles, lanterns, and *papel picado* at regular intervals. Figure 5 shows the layout.

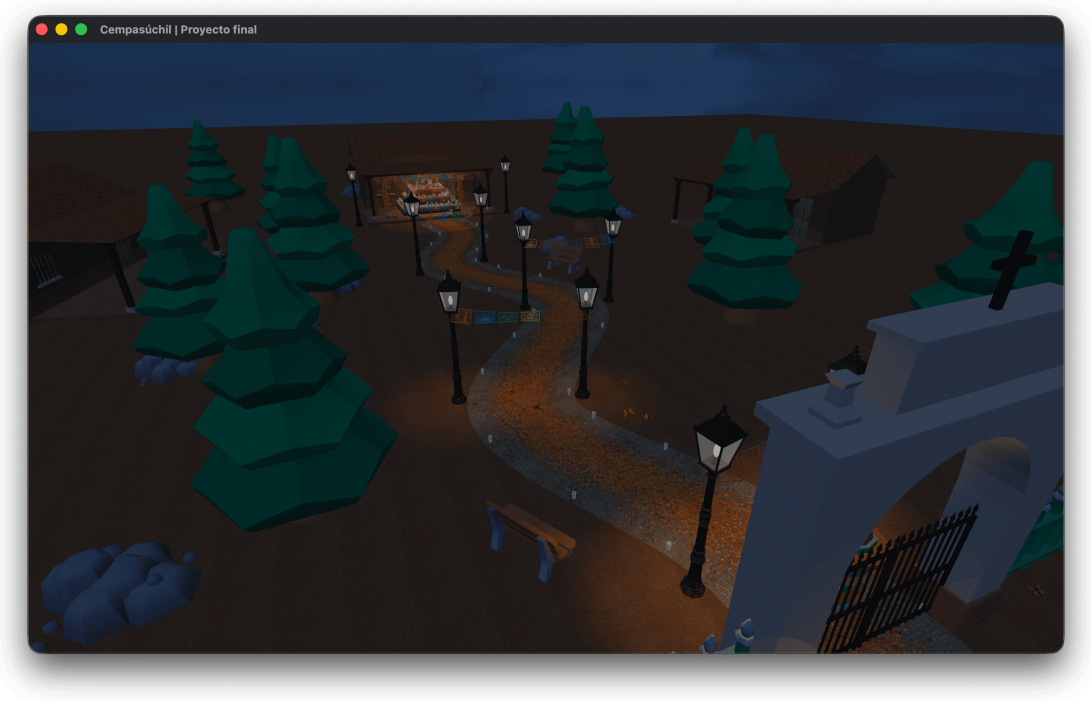


Figure 5: General view: cemetery, petal path, and the house with the offering.

4.6 Texturing, Color, and Lighting

The textures of the team-made models were designed with procedural nodes in Blender and baked to color, roughness, and normal maps. In the fragment shader, normals are perturbed without precomputed tangents: the `dFdx` and `dFdy` derivatives of position and UV coordinates give the tangent basis of each pixel. Petals and papel picado use alpha clipping to show their outline without extra geometry.

Lighting combines two techniques:

- **Phong** for opaque materials: a bluish directional moonlight and up to 25 point lights selected

among votive candles, lanterns, and altar fill lights, with quadratic attenuation. Specular sharpness is derived from material roughness.

- **Fresnel** for glass and water: Schlick's approximation mixes the transmitted color with a reflection that grows at grazing angles. This is what makes glasses, pitchers, and candle holders readable.

The environment is a skybox with a cube map. Its vertex shader removes the view translation so the horizon does not move, and its fragment shader darkens the original sunset toward a night tone. To avoid iterating over useless lights, the renderer keeps only the lights whose bounding volume can reach each object and culls primitives outside the camera using frustum planes extracted from the view-projection matrix (Gribb and Hartmann, 2001).

4.7 Animation

The specification asks for animations of three kinds (basic, keyframe, and procedural) and three media (code, external software, and bones). Table 5 relates each animation to its key, medium, and technique. All of them are coordinated by the `Animaciones` class (`src/scene/Animaciones.cpp`), which receives the key, advances each animation with the frame time, and writes the result into scene-graph nodes or renderer uniforms.

Key	Animation	Medium	Technique
1	Monarch butterfly	Code	State machine, keyframes on a Catmull-Rom path, and procedural flapping in the body-wing hierarchy.
2	Camera tour	Code	Seven position and look-at keyframes with smooth interpolation.
3	Votive candles	Code	On and off states; group-by-group ignition and procedural flicker in GLSL.
4	Papel picado	Blender	Shape keys baked into a 96-frame vertex cache.
5	Petal whirlwind	Blender	Keyframes of 720 petals exported as tracks.
6	Pitcher pouring water	Blender	Keyframes of the pitcher, water level, and stream exported as tracks.
7	Visitor 1	Bones	Seven-bone armature; walk, placing the offering, and bow.
8	Visitor 2	Bones	Second character; looks at a grave, waves, and dances.
9	Dancing skeleton	Bones	12 s dance with five blended steps.
L	Lanterns	Code	Light interaction: gradual switch off and on.

Table 5: Available animations and interactions.

4.7.1 Code Animations

Monarch butterfly (1). A butterfly rests on the petate in front of the altar. When the key is pressed it goes through the state machine rest, take-off, flight, and landing. The route is defined by ten control points that start at the resting spot, climb in front of the arch, and come back. It is interpolated with the same `Camino` class used for the path and traversed at constant speed. States are separated by the traveled distance: during the first and last 0.6 m the flapping ramps up or fades, and in the middle stretch it reaches full amplitude. The wings are child nodes of the body, so rotating them around their hinge is enough for them to follow the body's translation and orientation. This is an example of a transformation hierarchy.

Camera tour (2). Seven position keyframes and seven look-at keyframes describe the entry from the cemetery, the crossing of the gateway, the path, and the arrival at the offering. Both sequences are interpolated with Catmull-Rom and share a time parameter smoothed with $3t^2 - 2t^3$, so the camera starts and stops without jumps. While the tour runs, movement input is ignored; the same key stops it.

Votive candles (3). A C++ state machine goes through on, switching off, off, and switching on. Each state sets two values sent to the shader: an ignition front along the Z axis and a global level. In GLSL, the `factorVela` function groups candles into 4.4 m sections. A section lights up when the front reaches it, so candles turn on in groups from the cemetery toward the altar. The same factor adds procedural flicker: two sine waves with a pseudo-random phase per candle. The factor modulates both the flame emission and the point light each candle casts on its surroundings.

Lanterns (L). The second light interaction gradually brings the emission and light of the lanterns to zero and restores them when pressed again.

4.7.2 Animations Created in Blender

The three external-software animations were modeled and animated in Blender. For the engine they were baked into data that OpenGL plays back without depending on Blender.

Papel picado (4). Six sheets are deformed with shape keys (fixed edge, curvature, and wave) animated by keyframes with different phases and amplitudes. The result was baked into a 96-frame vertex cache; each frame the engine copies the positions into the vertex buffer. The key calms or restarts the wind: playback speed goes down or up gradually.

Petal whirlwind (5). In Blender, a script computed position and rotation keyframes for 720 petals: they lift off the path, swirl in a two-arm spiral in front of the altar, and settle with a sway. The `exportar_remolino.py` script reads the scene frame by frame and stores, for

each petal, its position, quaternion, and scale in the binary file `remolino_petalos.pist`. During the conversion it changes Blender axes (Z up) to OpenGL axes (Y up). In the engine the petals form an instanced batch. Each frame the two neighboring track frames are interpolated and the 720 matrices are rewritten with `glBufferSubData`.

Pitcher pouring water (6). The pitcher pivots around its spout. The water inside is clipped by a box whose height is animated so the surface stays level and the volume is preserved; the stream is a curve with an animated bevel; the glass fills through a shape key. The `exportar_jarra.py` script exports the pitcher's position and rotation, the water surface height, the glass fill level, and the stream factors. In OpenGL the water clipping becomes a plane: the fragment shader discards fragments with $\mathbf{n} \cdot \mathbf{p} + d > 0$. The stream is a tube whose rings are moved onto the exported curve every frame.

4.7.3 Skeletal Animations

The characters start from the pieces of the Kenney Graveyard Kit (Kenney, 2021), released under CC0. The `exportar_personajes.py` script builds in Blender a seven-bone armature (hips, torso, head, arms, and legs), joins the pieces into one mesh, and binds each piece to its bone. It then generates the clips (idle, walk, walk while carrying, place, stand up, bow, look, wave, short dance, and full dance) and exports each character as a glTF file with a skin and one animation per clip.

In the engine, `Modelo::posar` interpolates each bone's translation and rotation tracks (linear and spherical), computes their global matrices, and applies linear blend skinning:

$$\mathbf{v}' = \sum_i w_i \, G_i \, B_i^{-1} \, \mathbf{v}, \tag{1}$$

where G_i is the bone's global matrix, B_i^{-1} its inverse bind matrix, and w_i the vertex weight. The deformed vertices are uploaded to the mesh buffer and the bounding box is recomputed for frustum culling. Clips are blended over 0.25 s to avoid pose jumps.

Each character's trajectory is written in code as a sequence of segments (walk along a route, turn, play a clip). While walking, the cycle phase advances with the distance covered (0.95 m per cycle), which keeps the feet from sliding. Visitor 1 carries a pan de muerto attached to an anchor node that is a child of the forearm bone; when the placing clip ends, the bread stays on the first tier of the altar.

4.8 Navigation and Interaction

Navigation was designed as a first-person walk. At startup, the camera appears at the cemetery entrance, approximately 1.3 m above the ground and facing the altar. This position gives the visitor a visual objective from the first frame: lighting and the petal path show where to proceed without requiring an overlay interface.

In walking mode, WASD moves the visitor across the ground plane. Height remains fixed while a low-amplitude vertical and lateral bob is added. Its frequency and smoothness were adjusted to suggest footsteps without producing an aggressive oscillation. Shift temporarily multiplies the speed and the mouse wheel changes the base speed. Orientation follows mouse movement; TAB captures or releases the cursor, allowing the user to alternate between direct camera control and ordinary desktop use.

The F key enables free flight. Only in that mode are Q and E enabled for descending and ascending. This separation prevents the visitor from accidentally leaving ground level during normal walking, while preserving a useful tool for inspecting the scene, recording evidence, or reviewing objects from above. R returns the camera to the entrance and F2 toggles between the final finish and wireframe rendering.

In addition to free movement, key 2 starts a prepared camera tour as a demonstration. During that clip, movement input is suspended to prevent the user from displacing the trajectory; pressing the same key again cancels the route and immediately returns control. Table 6 summarizes all controls.

Input	Action
W, A, S, D	Walk through the scene.
Shift	Move faster.
Mouse and TAB	Look around; TAB captures or releases the mouse.
F, Q, E	Toggle free flight; go down and up while flying.
Mouse wheel	Adjust movement speed.
1 to 9	Start the animations in Table 5.
L	Switch the lanterns off or on.
F2	Toggle solid and wireframe rendering.
R	Return to the cemetery entrance.
Esc	Close the application.

Table 6: Application controls.

4.9 Integration of the Photographs

The seven picture frames display different photographs. To avoid duplicating seven complete glTF models, the engine loads three base frame models and creates logical variants of the photographic-surface material. Each variant replaces only that surface's texture while preserving the frame, geometry, maps, and remaining materials. Images were preprocessed as JPEG files with an orientation and crop compatible with the frame UVs, and are stored in `assets/textures/fotografias/procesadas/`.

This solution assigned a different person to every position without re-exporting the altar or changing the approved composition. Three portraits occupy the upper tier and four stand behind vases on the first and second tiers. Selecting the material by name keeps the change limited to the photograph and does not alter the wood, glass, or lighting of the frame.

5 Experiments

5.1 Building on Different Platforms

The project was built with CMake on macOS (Xcode), Windows (Visual Studio 2022 and 2026), and Linux (GCC). Tests in earlier deliveries found problems that remain solved in this version:

- Textures were missing outside macOS because image decoding only existed through ImageIO; `stb_image` was integrated for Windows and Linux.
- On scaled displays the logical window size differs from the framebuffer size. For that reason `glViewport` receives the size reported by `glfwGetFramebufferSize`.
- `Rutas.cpp` locates `assets/` and `shaders/` from the real executable path, with wide paths on Windows, so the working directory does not matter.
- Rounding differences between compilers change the offering piece count by a few units; a small difference is logged as a warning.
- The VirtIO driver of a Windows 11 ARM virtual machine did not expose OpenGL 3.3, so the application needs a driver with real OpenGL 3.3 support.

5.2 Automation and Distribution of the Windows Executable

To produce a reproducible delivery without requiring a team member to compile manually on Windows, the workflow `.github/workflows/windows-installer.yml` was added. It can be started manually from GitHub Actions and also runs whenever a tag beginning with `v` is published. GitHub assigns a Windows Server 2022 virtual machine, configures the project with CMake and Visual Studio 2022 for 64-bit architecture, builds the *Release* configuration, and verifies the executable, models, textures, animations, shaders, and instruction files.

Packaging is performed with CPack and NSIS. Installation rules in `CMakeLists.txt` place `CEMPASUCHIL.exe`, `assets/`, `shaders/`, `README.txt`, and the license in a single installer. GLFW and GLAD are incorporated during compilation, and the Visual C++ runtime is linked statically. The destination computer therefore does not need CMake, Visual Studio, Blender, or the repository; it only needs 64-bit Windows and a graphics driver supporting OpenGL 3.3 Core or later.

After the workflow finishes, GitHub Actions publishes the installer and its SHA-256 checksum as a temporary artifact. For a permanent download, release `v1.0.0`, titled *CEMPASÚCHIL v1.0.0 — Final Delivery*, was created in GitHub Releases. It contains `CEMPASUCHIL-Setup-0.2.0-win64.exe`, `SHA256.txt`, requirements, and complete usage instructions.

Click here to download the latest release of CEMPASÚCHIL

Installation consists of three short steps:

1. Download `CEMPASUCHIL-Setup-0.2.0-win64.exe` from the release's *Assets* section.
2. Run the installer and follow the wizard. Because this is an unsigned academic project, Windows may display a warning before opening it.
3. Start *CEMPASÚCHIL* from the Start menu or from the shortcut created by the installer.

The installer contains all scene resources and includes an uninstaller. The installed `README.txt` contains controls, animations, command-line options, and troubleshooting information. Users do not need to download the source code or copy resource folders separately.

5.3 Version Control and Work Integration

Version control was one of the most persistent challenges. The team worked on macOS, Windows, and Linux, and each operating system generated different build directories, projects,

caches, and temporary files. Early CMake, Xcode, and Visual Studio products occasionally became mixed with source files, increasing change size and creating conflicts unrelated to actual work.

The solution was to define a common structure and strengthen `.gitignore`: `build-*` directories, binaries, objects, logs, caches, Blender backups, and operating-system metadata were excluded. Model sources, final textures, export scripts, code, shaders, and documentation remained versioned. When local and remote histories diverged, `git pull –rebase` placed local work over the remote history, allowing conflicts to be resolved explicitly and avoiding accidental merge chains.

Resource size created another issue. Blender files, high-resolution textures, and animation caches do not behave like source code because Git cannot represent their differences efficiently. Before the final delivery, the team audited what the application actually loaded. Local builds, logs, duplicate photographs, unused models, and associated textures were removed, while the Blender sources needed to prove authorship and regenerate animations were retained.

5.4 Performance Problems and Optimization Decisions

The altar was the main performance pressure point: it concentrates thousands of petals, dozens of candles, flowers, food, transparent vessels, *papel picado*, and several light sources. The first implementation lost frames near the altar because dense meshes were combined with too many light evaluations per object. Static baked lighting was rejected because candles and lanterns needed to remain interactive in OpenGL.

Complementary measures were applied: polygon reduction on heavy models, normal maps to preserve apparent detail, instanced rendering, frustum culling, bounding volumes for light filtering, grouped altar lights, material caching, and buffer updates only while an animation requires them. The result preserves dynamic lighting and visual composition while avoiding evaluation of every light against every vertex or fragment.

5.5 Animation Validation

The animations were tested on Linux with Mesa (`llvmpipe`, OpenGL 4.5 Core) inside a virtual X server. A script outside the repository opened the application at a fixed view, sent the keys with `xdotool`, and took screenshots at intervals with FFmpeg. This way every animation was checked without manual input. The screenshots in the results section come from those runs.

The following was also verified:

- the exported tracks have the expected size: 264 frames $\times$ 720 petals $\times$ 8 values for the whirlwind, and 240 frames $\times$ 13 values for the pitcher;
- the pitcher starts 18 cm from the glass, tilts 60° while pouring, and the glass goes from 3% to 82% full;
- each character has seven joints and all its clips, with the frame length defined in Blender;
- visitor 1's bread rests in the front gap of the first tier, without intersecting the sugar skulls or the candles.

5.6 Performance

Since the engine already filtered lights by bounding volume and culled by frustum, the new animations were designed not to interfere with those mechanisms. The whirlwind updates a single instance buffer per frame. The characters are deformed on the CPU and have a few hundred vertices each. The pitcher only rebuilds the stream tube while it is visible. No animation adds a draw call per animated element.

6 Results

The final result of *CEMPASÚCHIL* is a walkable three-dimensional scene that brings together the different stages developed throughout the semester: planning, modeling, composition, texturing, lighting, animation, interaction, and optimization. The application stopped being only an isolated representation of an altar and became a complete environment composed of the cemetery, the petal path, the adobe house, and the *ofrenda*. This layout prevents the user from seeing every element at the beginning and instead allows the scene to be discovered while moving from the entrance toward the altar.

The final scene contains 2,134 nodes, 81 loaded models, and 10,549 instances. The difference between the number of models and the number of visible elements is mainly due to geometry reuse. Petals, flowers, votive candles, butterflies, walls, and other repeated objects share their meshes and are distributed through independent transforms. This made it possible to populate the environment without storing a complete copy of every element and without changing the visual composition that had already been approved.

The altar was the area that received the greatest amount of work. In addition to its three tiers, it incorporates votive candles, food, *pan de muerto*, sweets, drinks, vessels, *calaveritas de azúcar*, flowers, *papel picado*, vases, photographs, and different decorative objects. The placement of each piece had to be reviewed several times because the models came from different files and did not always share the same scale, orientation, or origin. In Blender, some objects looked correct individually, but after export they could appear too large, rotated, buried in a surface, or separated from their support point.

Solving this part required more than moving objects until they looked correct from a single camera. The team also checked that they did not intersect votive candles, that food rested on the tiers, that picture frames were not hidden by vases, and that the composition remained readable from different angles. The *ofrenda* ultimately functioned as a complete composition rather than an accumulation of independent models.

Integrating the photographs completed the visual purpose of the altar. The seven portraits correspond to Janis Joplin, Kurt Cobain, Valentín Elizalde, Amy Winehouse, Jim Morrison, Jimi Hendrix, and Kristen Pfaff. Instead of duplicating all picture-frame models, three base models were reused and material variants were generated to assign a different image to every position. This preserved geometry, frames, and their materials while modifying only the surface that displays the photograph.

During the texturing stage, the team confirmed that the appearance achieved in Blender did not transfer automatically to OpenGL. Several node networks relied on Blender-specific procedures that glTF could not reproduce directly. For this reason, procedural textures were baked into color, roughness, and normal maps. Baking preserved details such as ceramic irregularities, relief on *pan de muerto*, surface wear, and color variations without increasing geometry excessively.

UV coordinates, color spaces, transparency, and normal-map orientation also had to be reviewed. Some errors appeared only after export: dark borders around transparent areas, inverted textures, overly glossy materials, or images that did not load outside macOS. Integrating different image loaders and reorganizing resource paths allowed materials to be located in the same way on Windows, macOS, and Linux.

Final lighting remained entirely inside OpenGL. No static lights were baked into textures because votive candles and lanterns had to react to user interaction. The scene uses cool illumination to establish the nighttime environment and warm lights to mark the path and emphasize the *ofrenda*. The contrast between both areas helped make the altar the principal focal point without illuminating the entire environment uniformly.

The concentration of lights and models at the altar caused one of the main performance prob-

lems. Approaching this area combined thousands of instances, transparent surfaces, materials, and point sources. The solution was not to remove lighting, but to reduce unnecessary work. Heavy models were optimized, normal maps preserved detail, nearby lights were grouped, sources were filtered through bounding volumes, and objects outside the camera were discarded. These changes preserved the appearance of the scene and reduced frame loss during the route.

Navigation also changed from the earliest deliveries. The camera stopped functioning only as a free camera and began simulating a first-person journey. The user starts at the cemetery entrance, at approximately human height, and can walk using WASD. Movement includes a light bob so that displacement does not feel completely rigid. Flight remains available as a separate option for inspecting the scene, but during walking the user cannot accidentally move up or down.

The animation stage brought together three different forms of work. The butterfly, camera route, and votive-candle animations were developed directly in code. *Papel picado*, the petal whirlwind, and the pitcher were created in Blender and then exported as data OpenGL could reproduce. Finally, the visitors and skeleton used armatures and skeletal animation. The scene therefore contains nine principal animations controlled by keys 1 through 9, in addition to the lantern on/off interaction.

The *papel picado* animation was one of the most complicated integrations because a conventional export did not preserve exactly the deformation observed in Blender. The movement was solved with a 96-frame vertex cache. OpenGL reproduces the exported positions and keeps the upper edge attached to the cable fixed. This allowed every sheet to move differently and prevented all pieces from sharing an artificial deformation.

The petal whirlwind uses 720 animated elements. Instead of creating one draw call per petal, one instanced batch is updated. The pitcher required coordination among vessel rotation, water level, stream, and glass filling. Characters required loading bones, clips, weights, and transformation matrices, as well as blending poses to avoid abrupt changes among walking, stopping, placing an offering, waving, or dancing.

Another important result was cross-platform compatibility. The project was developed on macOS, Windows, and Linux computers, which introduced differences in compilers, paths, libraries, and graphics drivers. Some failures appeared to belong to the scene but were actually caused by how each system located resources or created the OpenGL context. CMake configuration, image loaders, and platform launchers helped preserve one project without dividing the code into incompatible versions.

Version control also became part of development. The repository came to contain large

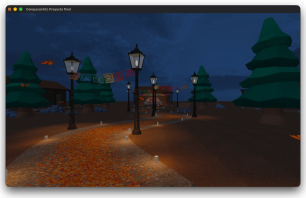
Blender models, textures, screenshots, documentation, and animation caches. Conflicts did not occur only in code; they also appeared when two members modified composition files, documentation, or binary resources. Branches had to be reorganized, remote changes integrated through *rebase*, generated folders excluded, and the files truly required for delivery reviewed.

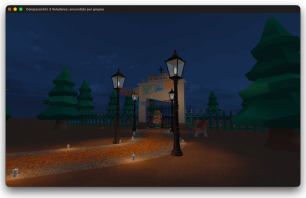
Final cleanup reduced duplicate files, local builds, logs, models no longer present in the scene, and photograph versions unused by the program. At the same time, source files required to demonstrate modeling and regenerate animations were preserved. After cleanup, a build from scratch verified that no deleted resource was essential.

Overall, the result meets the requirements for original and library models, texturing, skybox, light interaction, code animation, external animation, skeletal animation, and user input. Beyond the number of elements, the principal result was making work produced at different moments and in different tools operate as one scene. Blender modeling, baked textures, OpenGL lighting, animation tracks, and navigation were integrated into an application that can be explored and demonstrated as a complete product.

7 Visual Evidence of the Final Scene

The following screenshots document the final version of CEMPASÚCHIL. They are grouped by composition, detail, lighting, and animation in order to show both the visual result and state changes that cannot be appreciated in a single image.

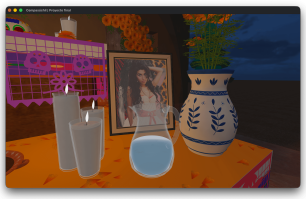

Cemetery gateway

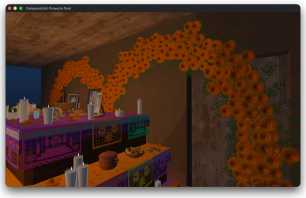

Initial view

Cempasúchil path

Final ofrenda

Food models

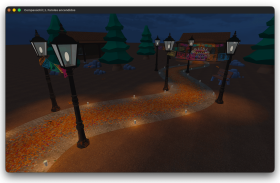

Vessels and glass

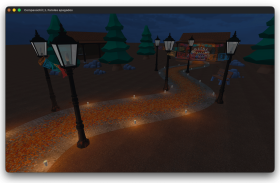

Floral arrangement

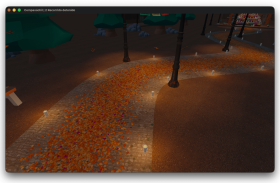
Papel picado

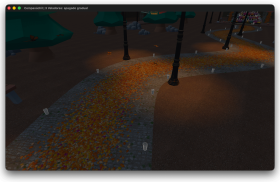
Picture frames

Figure 6: General views and model details. The route connects the entrance, path, and ofrenda. Close views show baked textures, normal maps, transparency, and the photographs on the altar.

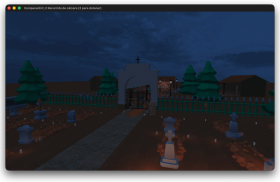

Lanterns on

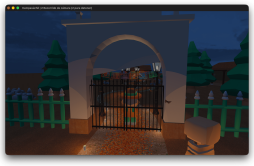

Lanterns off


Votive candles on


Votive candles off

Monarch butterflies


Route start


Gateway

Path

House

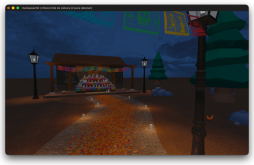

Arrival at the altar

Figure 7: Interactive lighting and animations programmed in OpenGL. Key L changes lanterns and votive candles. Butterflies combine Catmull–Rom with a body–wing hierarchy, while the automatic camera travels to the altar.

Papel picado at rest

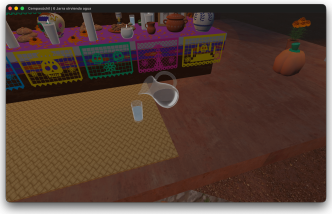
Wind deformation

Whirlwind start

Petal lift

Formed spiral

Beginning of the pour



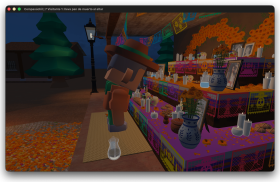
Stream and filling glass

Figure 8: Animations created in Blender. Exported caches and tracks preserve the papel picado deformation, the petal whirlwind, and the pitcher motion with the changing water level.

Arrival at the altar

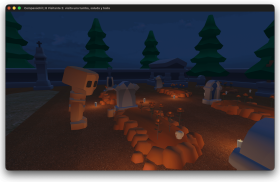
Preparation

Placing the offering



Final bow

Route among graves



Observing a grave

Visitor dance

First skeleton step

Second skeleton step

Figure 9: Skeletal animations. The clips show the visitor placing a pan de muerto, the character walking through the cemetery, and the skeleton dancing beside the gateway.

8 Demonstration Video

A demonstration video was produced for the final delivery. It presents the complete journey through the scene, principal models, dynamic lighting, navigation controls, and the nine integrated animations.

The video shows the three animations developed through code, the three animations created in Blender, and the three skeletal animations. It also includes the interaction used to switch the path lanterns on and off.

Figure 10: CEMPASÚCHIL demonstration video. Click the image to open the video on YouTube.

Click here to watch the demonstration video

`https://youtu.be/2iYnnTLFxNo`

9 General Conclusions

CEMPASÚCHIL demonstrates that an academic scene can grow incrementally without discarding its earliest decisions. The primitive-based altar served as a planning tool; the modeling stage turned that idea into a walkable environment; texturing and lighting established a vi-

sual identity; and animation transformed static objects into events the visitor can activate and observe.

The main lesson was not one isolated technique, but integration. A model that looks correct in Blender still needs appropriate scale, origin, UVs, exportable materials, and reasonable cost before it works in OpenGL. An attractive light can become too expensive when multiplied across thousands of instances. An animation that is convincing in external software requires a decision about which data to export and how to reproduce them. An application that works on one computer needs portable paths, decoders, and launchers before it becomes a reproducible delivery.

Optimization did not mean removing detail indiscriminately. Dynamic lighting, translucent materials, photographs, the visual density of the altar, and all animations were preserved, but their computation changed: instances instead of copies, normal maps instead of unnecessary geometry, filtered lights instead of global evaluation, and compact tracks instead of requiring Blender during execution.

Beyond the technical result, the portraits of Janis Joplin, Kurt Cobain, Valentín Elizalde, Amy Winehouse, Jim Morrison, Jimi Hendrix, and Kristen Pfaff turn the altar into a place of remembrance. The journey does not end only at the zone with the most models or lights; it ends before seven brief lives whose work continues to accompany other people. That permanence connects the project title, the cempasúchil path, and the movement of the final scene.

10 Individual Conclusions

Basilio Illescas Andrés

At first, the project appeared solvable by building an altar and placing a few representative objects, but the prototype showed that distribution would be as important as model detail. Working first with primitives helped define the three tiers and identify which objects needed more space. When the proposal expanded toward the cemetery, path, and house, the organization of the work also had to change because the task was no longer one composition but several connected areas.

One of the main challenges was maintaining a shared visual direction while models were developed separately. Each object could look correct in Blender, yet differences in scale, amount of detail, and style appeared when they were assembled. The altar proportions, spacing between objects, and readability of the scene from the route had to be reviewed several times. Planning did not end with the first delivery; it had to be adjusted whenever new models, ani-

mations, or performance problems appeared.

From the Product Owner role, the most important part was deciding which elements truly contributed to the concept and which only added complexity. The number of objects grew considerably, but the team tried to preserve a recognizable route and keep the altar as the principal focus. It was also important to ensure that partial deliveries did not remain separate projects, but that each one served as the foundation for the next.

The final result preserves the idea of the first prototype even though it is visually much larger and more detailed. The three-tier structure, warm light, and portraits remain at the center of the proposal. I consider the greatest achievement to be transforming a simple idea into a complete environment without losing the original purpose of the *ofrenda*. This project also made it clear that planning does not mean fixing everything at the beginning, but continuously reviewing whether what is being built still works as a whole.

Cruz Macedo Samuel Santiago

My work focused mainly on architectural and environmental elements, so an early challenge was making the cemetery, entrance, path, and house feel like parts of one place. Team-made and library models introduced scale and orientation differences from the first integration. Importing them was not enough: their positions, the terrain, and their appearance from the actual camera height had to be reviewed.

Compatibility among Windows, Linux, and the other team computers also caused several problems. In some cases, the project compiled but could not find textures or shaders because it was being executed from a different directory. At other times, the difficulty involved Visual Studio, CMake versions, or OpenGL drivers. This required separating errors in the code from errors caused by the execution environment itself.

Working with GitHub was another important part. When handling models, textures, screenshots, and documentation, changes could be much heavier than in a project formed only by source code. Differences also appeared between local and remote versions, along with integration conflicts and files generated by each operating system that did not belong in the repository. I learned that using Git well does not consist only of making commits, but of keeping changes clear, reviewing repository state before synchronizing, and avoiding files that can be regenerated.

Testing on different platforms helped identify assumptions that were invisible on a single computer. Paths, image formats, window creation, and OpenGL availability could change even when the code was the same. I believe this improved the stability of the project because it

forced us to prepare an application that depended less on one specific computer. The result was not only a complete scene, but a project that can be built and reviewed outside the environment in which it was originally developed.

Pérez Olvera Alexis Abraham

A considerable part of my work focused on models and objects for the *ofrenda*. At first the goal was to meet the required model count, but as the altar filled, it became more important to review how the objects worked together. Some hid others, intersected the candles, or failed to rest correctly on the tiers. The final composition required several adjustments to distribute food, vessels, flowers, portraits, and decorations without losing clarity.

Texturing presented difficulties different from modeling. Materials that looked correct in Blender could not always be exported in the same way, especially when they used procedural nodes. Texture baking transferred part of that detail to OpenGL, but it also required reviewing UVs, resolutions, color spaces, and normal maps. In some cases it was necessary to return to the model, correct its mapping, and export it again before obtaining an acceptable result.

Lighting completely changed the perception of materials. A texture could look correct under Blender lighting and appear too dark or glossy in the application. Glass and water required treatment different from opaque materials, while votive candles and lanterns had to illuminate the scene without making rendering too expensive. This made it clear that modeling, texturing, and lighting cannot be reviewed independently.

Photograph integration was one of the stages that gave the altar its final meaning. Having seven different images without duplicating all picture-frame geometry allowed the organization to remain cleaner. The crop and orientation of each image also had to be controlled so it appeared correctly on the model surface. Looking at the complete *ofrenda*, I believe the detailed work helped the objects stop feeling like separate tests and begin to form a scene with intention.

Trejo Olvera Emmanuel

My work focused mainly on overall integration, OpenGL development, optimization, and testing on macOS. A recurring problem was that something could work correctly in Blender or in an isolated test but not necessarily inside the complete scene. Integrating a model meant checking its path, materials, scale, and bounding box; integrating an animation also required

coordinating time, transforms, and buffer updates without affecting the rest of the rendering process.

On macOS, several issues involved execution paths, Retina display scaling, image loading, and launcher behavior. In some cases, the window was created correctly but the rendered area did not match the real framebuffer size. It was also necessary to consider that Finder, the terminal, and a packaged application do not always use the same working directory. This led to improved resource discovery and eliminated dependence on paths relative to the folder from which the program was launched.

Animation integration was probably the most complex part. The butterfly and camera route could be controlled directly through code, but *papel picado* had to preserve the movement created in Blender. Early approaches did not produce the same result: the sheets moved uniformly or deformed from incorrect regions. Exporting a vertex cache kept the upper edge fixed and reproduced each sheet's deformation. The whirlwind, pitcher, and characters presented similar challenges because each required different information to be represented inside the engine.

Performance also became a problem when reaching the altar. The number of instances, transparent surfaces, and lights caused a frame-rate loss that was not noticeable in other areas. The team did not want to remove lighting or reduce the composition excessively, so work focused on filtering lights, reusing geometry, discarding elements outside the camera, and optimizing heavy models. This required repeated testing of the complete scene because a local improvement could produce a visual change in another material or animation.

Finally, remote integration and GitHub required care with large files, branches, and stable versions. There were moments when documentation, code, and resources advanced simultaneously, making it easy to mix changes or attempt to upload generated files that were too heavy. Maintaining a stable branch, reviewing every integration, and cleaning the project before packaging were as important as implementing the graphics features. What I take from the project is the understanding that a scene of this size is not complete merely when it looks good, but when it can also be built, executed, documented, and delivered consistently.

11 References

- Akenine-Möller, T., Haines, E., Hoffman, N., Pesce, A., Iwanicki, M., & Hillaire, S. (2018). *Real-time rendering* (4th ed.). CRC Press.
- Barry, P. J., & Goldman, R. N. (1988). A recursive evaluation algorithm for a class of Catmull-Rom splines. *ACM SIGGRAPH Computer Graphics, 22*(4), 199–204. https://doi.org/10.1

145/378456.378511

- Blender Foundation. (2026). *Blender 5.2 manual*. https://docs.blender.org/manual/en/latest/
- Catmull, E., & Rom, R. (1974). A class of local interpolating splines. In R. E. Barnhill & R. F. Riesenfeld (Eds.), *Computer aided geometric design* (pp. 317–326). Academic Press.
- Gregory, J. (2018). *Game engine architecture* (3rd ed.). CRC Press.
- Gribb, G., & Hartmann, K. (2001). *Fast extraction of viewing frustum planes from the world-view-projection matrix*. https://www.gamedevs.org/uploads/fast-extraction-viewing-frustum-planes-from-world-view-projection-matrix.pdf
- Kenney. (2021). *Graveyard kit* [3D models, CC0 license]. https://kenney.nl/assets/graveyard-kit
- Khronos Group. (2021). *glTF 2.0 specification*. https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html
- Magnenat-Thalmann, N., Laperrière, R., & Thalmann, D. (1988). Joint-dependent local deformations for hand animation and object grasping. In *Proceedings of Graphics Interface '88* (pp. 26–33). Canadian Information Processing Society.
- Phong, B. T. (1975). Illumination for computer generated pictures. *Communications of the ACM, 18*(6), 311–317. https://doi.org/10.1145/360825.360839
- Schlick, C. (1994). An inexpensive BRDF model for physically-based rendering. *Computer Graphics Forum, 13*(3), 233–246. https://doi.org/10.1111/1467-8659.1330233
- Segal, M., & Akeley, K. (2010). *The OpenGL graphics system: A specification (version 3.3, core profile)*. Khronos Group. https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf
- Shoemake, K. (1985). Animating rotation with quaternion curves. *ACM SIGGRAPH Computer Graphics, 19*(3), 245–254. https://doi.org/10.1145/325165.325242
- UNESCO. (2008). *Indigenous festivity dedicated to the dead*. Intangible Cultural Heritage. https://ich.unesco.org/en/RL/indigenous-festivity-dedicated-to-the-dead-00054
- Yuksel, C., Schaefer, S., & Keyser, J. (2011). Parameterization and applications of Catmull-Rom curves. *Computer-Aided Design, 43*(7), 747–755. https://doi.org/10.1016/j.cad.2010.08.008